Phase 1, Part 2: Develop the Financial Data Tool (yfinance Integration)

Now that your project structure is in place and environment variables are loading, let's build the first core tool for your agent: the Financial Data Tool. This tool will leverage the `yfinance` library to fetch real-time and historical stock data, which is crucial for any financial analysis.

Step 1: Install `yfinance`

First, ensure your virtual environment is activated and install the `yfinance` library.

Bash

```
source venv/bin/activate # On Windows, use `venv\Scripts\activate`
pip install yfinance pandas
```

We also install `pandas` as `yfinance` often returns data in pandas DataFrames, which are excellent for data manipulation.

Step 2: Create the Financial Data Tool in `tools.py`

Open your `tools.py` file and add the following Python code. This code defines a function `get_stock_info` that takes a stock ticker symbol as input and returns relevant financial data.

Python

```
# tools.py

import yfinance as yf
import pandas as pd
from datetime import datetime, timedelta

def get_stock_info(ticker: str) -> str:
    """
    Fetches current and historical stock information for a given ticker symbol.
    Returns a string summary of the stock's performance.
    """
    try:
        stock = yf.Ticker(ticker)
        info = stock.info

        # Get current price and basic company info
        current_price = info.get('currentPrice', 'N/A')
```

```
        company_name = info.get('longName', ticker)
        sector = info.get('sector', 'N/A')
        industry = info.get('industry', 'N/A')
        market_cap = info.get('marketCap', 'N/A')

        # Fetch historical data for the last 7 days
        end_date = datetime.now()
        start_date = end_date - timedelta(days=7)
        hist = stock.history(period="7d")

        if not hist.empty:
            # Get open, close, high, low for the last trading day
            last_day_data = hist.iloc[-1]
            last_open = last_day_data['Open']
            last_close = last_day_data['Close']
            last_high = last_day_data['High']
            last_low = last_day_data['Low']

            # Calculate 7-day performance
            seven_day_change = (hist['Close'].iloc[-1] - hist['Close'].iloc[0])
            seven_day_summary = f"7-day change: {seven_day_change:.2f}%"
        else:
            last_open, last_close, last_high, last_low = 'N/A', 'N/A', 'N/A', '
            seven_day_summary = "No historical data for the last 7 days."

        summary = f"Company: {company_name} ({ticker})\n"
        summary += f"Sector: {sector}, Industry: {industry}\n"
        summary += f"Market Cap: {market_cap:,}\n"
        summary += f"Current Price: {current_price}\n"
        summary += f"Last Trading Day (Open: {last_open}, Close: {last_close},
        summary += seven_day_summary

        return summary

    except Exception as e:
        return f"Error fetching stock info for {ticker}: {e}"


# Example usage (for testing purposes, can be removed later)
if __name__ == "__main__":
    apple_info = get_stock_info("AAPL")
    print(apple_info)

    google_info = get_stock_info("GOOGL")
    print(google_info)

    invalid_info = get_stock_info("INVALIDTICKER")
    print(invalid_info)
```

Step 3: Test Your Financial Data Tool

To ensure your `get_stock_info` function works as expected, run `tools.py` directly from your terminal:

Bash

```
python tools.py
```

You should see output similar to this (values will vary based on real-time data):

Plain Text

```
Company: Apple Inc. (AAPL)
Sector: Technology, Industry: Consumer Electronics
Market Cap: 3,200,000,000,000
Current Price: 200.0
Last Trading Day (Open: 198.0, Close: 201.0, High: 202.0, Low: 197.0)
7-day change: 1.50%

Company: Alphabet Inc. (GOOGL)
Sector: Communication Services, Industry: Internet Content & Information
Market Cap: 2,100,000,000,000
Current Price: 175.0
Last Trading Day (Open: 174.0, Close: 176.0, High: 177.0, Low: 173.0)
7-day change: 0.80%

Error fetching stock info for INVALIDTICKER: No data found, symbol may be delis
```

This output confirms that your `yfinance` integration is working, and your `get_stock_info` function can successfully retrieve and summarize financial data. This tool will be integrated into your LangChain agent in a later phase.